

Predicting Survival of Critically Ill Patients: an Hist-GradientBoost approach

Elena Perotti
Politecnico di Torino
Student id: s345824
s345824@studenti.polito.it

Abstract—In this report, an approach to the ill-patient survival classification problem is presented. The proposed method consists of a preprocessing phase for dataset cleaning and feature analysis, followed by feature extraction based on the models implemented. Finally, these models have been tested together with different feature subsets, are then compared through cross-validation, and the configuration achieving the best macro F1-score is selected as the final solution.

I. PROBLEM OVERVIEW

The dataset consists of an ensemble of critically ill patients admitted to five US medical centers, and the goal is to determine whether each of them survived or not in a specific time frame. In this classification problem the target (*death*) is binary and takes value 1 when the patient is deceased, 0 when the patient survived. The dataset is divided into two main parts:

- a *development set*, containing 7284 records, for which the binary label is provided;
- an *evaluation set*, containing 1821 records, for which the binary label is not provided and has to be predicted.

The dataset is composed of 46 columns (excluding the target label and the row identifier), including numerical, binary, categorical and ordinal features. We can say that we have a lot of informations about teh patients:

- *age*, *sex*, *edu* (years of education), *income* (financial situation) and *race* are personal information about the patients;
- *dzgroup*, *dzclass*, *num.co* indicate the type of disease the patient has, and how many diseases he has simultaneously;
- *surv2m*, *surv6m*, *prg2m*, *prg6m*, *scoma*, *sps* are all useful features predicted by another model about survival;
- *medical_center*, *admission_month*, *charges*, *totcst* and *totmct* are informations about the hospital and their charges;
- *aps* and *avtisst* are two additional day-3 severity/intensity-of-care scores;
- *hday*, *dnr* and *dnrday* describe administrative and end-of-life-care aspects of the hospital stay;
- *diabetes*, *dementia* and *ca* (cancer status) report chronic comorbidities of the patient;
- *meanbp*, *wbhc*, *hrt*, *resp*, *temp*, *temp_f*, *pafi*, *alb*, *bili*, *crea*, *sod*, *ph*, *glucose*, *bun* and *urine* are the day-3 vital signs and blood/laboratory measurements;

- *adlp*, *adls* and *adlsc* are Activities of Daily Living indices measuring the patient’s functional autonomy;

We can also notice that the target is unbalanced: class 0 (patient alive) accounts for 31.9% of the development set, while class 1 (deceased patient) accounts for the remaining 68.1%, as shown in Fig. 1. No duplicated rows are present in the development set.

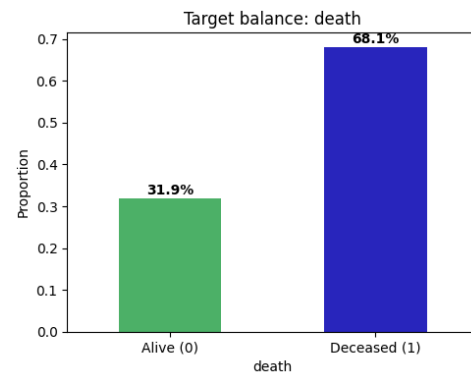


Fig. 1. Target balance of the development set.

II. PROPOSED APPROACH

A. Preprocessing

- **Missing Values:** Missing values also present in several columns: some features, such as *adlp*, *urine* and *glucose*, are missing in 50-60% of the rows, while most physiological measurements (e.g. *hrt*, *resp*, *sps*, *aps*) have no missing values. At the row level, however, no patient has more than about 40% of its features missing, so no records were removed for excessive sparsity.
- **Data cleaning:** no duplicated rows were found in the development set, so the cleaning phase focused on removing uninformative columns and handling missing values. The *Id* column is just a row identifier and carries no predictive information, so it is dropped unconditionally. The *temp* and *temp_f* columns encode the same measurement (body temperature) in Celsius and Fahrenheit, so *temp_f* is redundant and is dropped as well, keeping only *temp*.

- **Feature analysis:** For the elevate number of features, in this part we will analyze them to see if there could be useless, and test after if their absense improve our performance.

1) *Correlation with the target:* Fig. 2 reports the absolute Pearson correlation of each numerical feature with *death*.

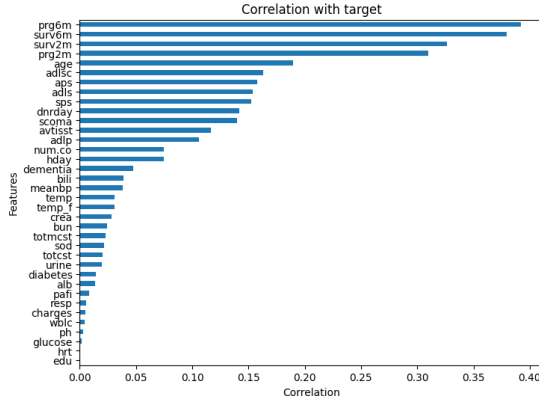


Fig. 2. Absolute correlation of numerical features with the target *death*.

As expected, the SUPPORT model's own survival estimates (*prg6m*, *surv6m*, *surv2m*, *prg2m*) are the most correlated features, since they are themselves predictions of a similar outcome produced by another model. The correlation graph also shows that a lot of features have a really low correlation with the target. In particular *edu* (years of education) is the lowest, with a value of ≈ 0.000125 ; for this reason, candidated with a really low correlation value has been considered as candidate to delete and tested in modeling process.

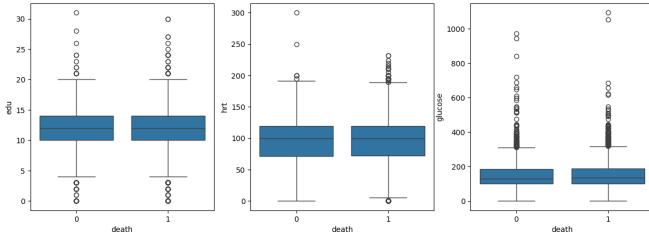


Fig. 3. Distribution of *edu*, *hrt* and *glucose* split by target class; *edu* in particular shows almost perfectly overlapping distributions between the two classes.

2) *Redundancy between features:* It has been noticed that several features has strongly correlated pairs:

- *adls* / *adlsc* have a correlation of ($\rho = 1.00$). Moreover, there also feature *adlp* which is the same feature compiled by the patient. It can be biased because it has a lot of missing values and sometimens the ill patient can make mistakes.
- *surv2m* / *surv6m* ($\rho = 0.96$) and *prg2m* / *prg6m* ($\rho = 0.90$) are highly correlated, which makes sense because they predict the same value with a

prevision of 2 months and 6 months. However, I decided to keep them because they are both highly informative.

- *totcst* / *totmcs* ($\rho = 0.94$) is also correlated, so it has been tested if removing one of them improve performance.

It has also been verified that *dzgroup* (8 categories) fully determines *dzclass* (4 categories, Fig. 4), i.e. the two variables are redundant with each other. It has been tried to remove *dzclass* from the dataset.

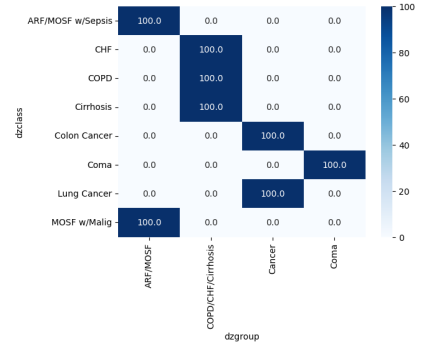


Fig. 4. Row-normalized cross-tabulation of *dzgroup* vs. *dzclass*: each disease sub-category maps to exactly one disease category.

- **Feature Extraction** After the cleaning step, the remaining features are split into three groups and handled through a ColumnTransformer. Two encoding variants were implemented, one for each family of models used in Section II-B:

- **Classic encoding** (used for Logistic Regression): numerical features are median-imputed and standardized (zero mean, unit variance); ordinal features (*income*, *ca*, *num.co*) are most-frequent-imputed and ordinally encoded respecting the natural order of the categories; nominal features are most-frequent-imputed and one-hot encoded;
- **HistGradientBoosting encoding:** numerical features are passed through unchanged, including their missing values, which the model handles internally by learning the best split direction for missing entries at each node; ordinal and nominal features are integer-encoded with an ordinal encoder instead of one-hot encoded, avoiding the increase in dimensionality that one-hot encoding would cause. The categorical columns are additionally declared to the classifier through its `categorical_features` argument, so HistGradientBoosting uses native categorical splits on them instead of treating the encoded integers as an arbitrary order.

B. Model selection

Two model were compared:

- **Logistic Regression:** a linear model that estimates the probability of the positive class by applying a sigmoid function to a weighted sum of the input. It is fast to train,

easy to interpret (each coefficient reflects the direction and strength of the association between a feature and the outcome), and serves as a well-calibrated baseline;

- **HistGradientBoosting**: a histogram-based gradient boosting ensemble of shallow decision trees built sequentially, where each new tree is trained to correct the errors (residuals) of the ensemble built so far. Feature values are pre-binned into histograms before the search for the best split, which makes training substantially faster than a standard gradient boosting implementation, and the model natively supports missing values and, in principle, categorical features. Unlike a linear model, it can capture non-linear relationships and interactions between features.

Ensemble methods based on boosting are generally reported to reach a better score-vs-training-time trade-off than bagging-based Random Forests [1], since the latter tends to plateau earlier as more trees are added. Moreover, HistGradientBoosting was preferred over the standard GradientBoostingClassifier because it natively handles missing values and offers a built-in `class_weight` option, which is directly useful given the class imbalance discussed in Section II-A [2].

Each classifier was evaluated using different combinations of features, as described in the preprocessing section:

- **full**: all the retained features (all columns after dropping *Id* and *temp_f*);
- **reduced_edu**: the *full* configuration without *edu*, whose correlation with the target is essentially null;
- **reduced_adls+adlp**: the *full* configuration without the two ADL indices *adlp* and *adls*, keeping only the calibrated version *adlsc*;
- **reduced_hrt+edu**: the *full* configuration without *edu* and *hrt* (heart rate), another feature with a correlation close to zero (Fig. 2);
- **reduced_dzclass**: the *full* configuration without *dzclass*, which Fig. 4 showed to be fully determined by *dzgroup*;
- **reduced_totcst**: the *full* configuration without *totcst*, whose near-duplicate *totmcst* ($\rho = 0.94$) is kept;
- **reduced_all**: the *full* configuration without *edu*, *hrt*, *totcst*, *dzclass*, *adls* and *adlp* simultaneously, combining all of the reductions above.

All configurations were evaluated through 5-fold stratified cross-validation (used StratifiedKFold to keep the same proportion of target data in every fold, because the dataset is not balanced), using **macro F1-score** as the scoring metric, consistently with the evaluation metric of the challenge.

C. Hyperparameters tuning

For each combination of preprocessing configuration and model, an exhaustive grid search (GridSearchCV) was performed over the following hyperparameter grids:

- **Logistic Regression**: regularization strength $C \in \{0.01, 0.1, 1, 10\}$;
- **HistGradientBoosting**: maximum number of boosting iterations $\in \{100, 200\}$, learning rate $\in \{0.05, 0.1\}$, maximum tree depth $\in \{\text{None}, 6, 10\}$.

The default decision threshold (0.5) is used throughout for both models; a decision-threshold search was considered but was left out of scope for this submission, relying on the class-weight reweighting alone to address the imbalance.

III. RESULTS

Table I reports, for each configuration and model, the best cross-validated macro F1-score obtained by the training set and the score obtained on trinidad platform for the test set:

TABLE I
CROSS-VALIDATION RESULTS (MACRO F1-SCORE) FOR ALL TESTED CONFIGURATIONS.

Config	Model	Train score	Test score
full	HistGB	0.7513	0.735
reduced_edu	HistGB	0.7527	0.748
reduced_adls+adlp	HistGB	0.7514	0.744
reduced_hrt+edu	HistGB	0.7486	0.744
reduced_dzclass	HistGB	0.7510	0.740
reduced_totcst	HistGB	0.7543	0.740
reduced_all	HistGB	0.7515	0.731
full	LogReg	0.7332	0.727
reduced_edu	LogReg	0.7325	0.727
reduced_adls+adlp	LogReg	0.7332	0.722
reduced_hrt+edu	LogReg	0.7328	0.723
reduced_dzclass	LogReg	0.7328	0.731
reduced_totcst	LogReg	0.7337	0.729
reduced_all	LogReg	0.7336	0.721

Across all configurations, HistGradientBoosting outperforms Logistic Regression, confirming that the two classes are only moderately separable with the available features but that the non-linear boosting model extracts a meaningful additional signal over the linear baseline. This modest gap is consistent with the broader clinical-prediction literature: in a systematic review of 71 studies comparing logistic regression with more flexible machine learning models, Christodoulou et al. [3] found no systematic performance advantage for machine learning, attributing this to the fact that clinical prediction problems typically have a low signal-to-noise ratio. A small but consistent HistGradientBoosting/LogReg gap, rather than a large one, is therefore the expected outcome if most of the available signal in this dataset is already linearly recoverable.

The best configuration on trinidad website is

- **HistGradientBoosting** on the **reduced_edu**:
{`class_weight`='balanced', `learning_rate`=0.1,
`max_depth` = *None*, `max_iter` = 100}

whereas the best configuration on our training set is

- **HistGradientBoosting** on the **reduced_totcst**:
{`class_weight`='balanced', `learning_rate`=0.05,
`max_depth` = *None*, `max_iter` = 100}

The first one reaches a test macro F1-score of **0.748**, the highest among all evaluated configurations. Its cross-validated score (0.7527) is, however, only the second-highest among the seven HistGradientBoosting configurations: *reduced_totcst*

scores marginally higher in cross-validation (0.7543) but generalizes slightly worse on the leaderboard (0.740).

The fact that the best results between our training and the result on the test set can be explained in 2 ways:

- 1) the small differences between configurations are within the noise of a single train/test split rather than reflecting a true difference in generalization;
- 2) HistGradientBoosting, as a tree-based model, can largely ignore uninformative or noisy features during training by simply not splitting on them, so removing them helps only marginally. That's why our results improve not that much using a reduced number of features;

IV. DISCUSSION

Fig. 5 shows the **ROC curve** of the best model (*reduced_totcst*): we can notice that threshold = 0.5 seems to be the best choice, because it has the best balance between False positive rate and True positive rate. The model reaches an AUC of 0.843.

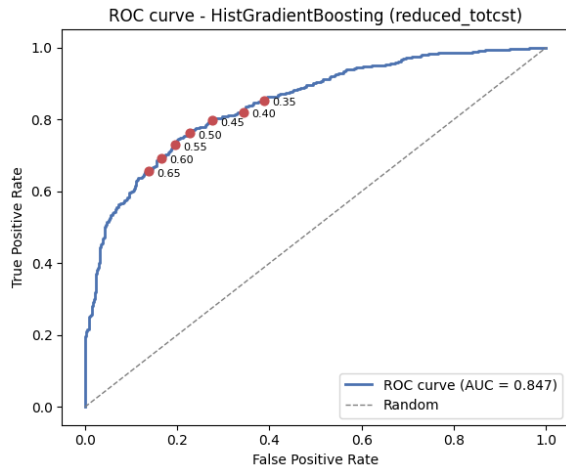


Fig. 5. ROC curve of the selected model (HistGradientBoosting, *reduced_edu*) on the validation split.

Fig. 6 shows, instead, the **Confusion matrix**: we can notice that the model correctly identifies 77% of the alive patients in the validation split (recall = 0.77) and 76% of the deceased patients (recall = 0.76): thanks to `class_weight='balanced'`, the two per-class recalls are almost identical – if anything, recall on the minority *Alive* class is marginally higher than on *Deceased* – so the model is clearly not defaulting to the majority class. Since the matrix is row-normalized, it does not show precision directly, but it can still be inferred qualitatively: with a comparable false-positive rate on each class (23% of alive patients misclassified as deceased, 24% of deceased patients misclassified as alive) and deceased patients being roughly twice as frequent as alive ones in the development set (68.1% vs. 31.9%, Section II-A), the absolute number of alive patients wrongly labeled as deceased is still larger than the reverse. This means precision on the *Alive* class remains the model's clearest weak point, even though recall is well balanced across both classes.

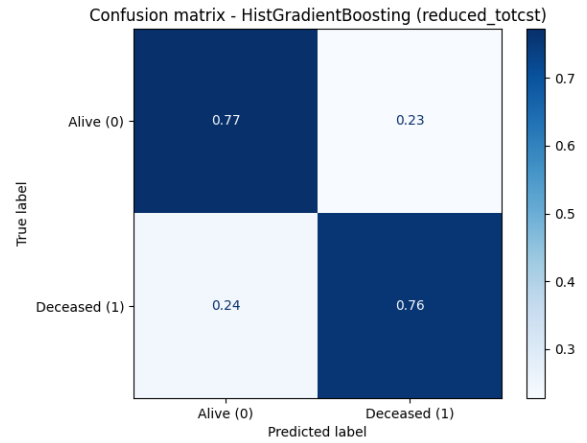


Fig. 6. Confusion matrix of the selected model on the validation split (default threshold = 0.5).

Overall, given the intrinsic difficulty of predicting an individual patient's survival – an outcome that depends on a very large number of interacting physiological, demographic and treatment-related factors, many of which are not captured by the available features or are only proxies for the patient's true clinical trajectory; a macro F1-score around 0.75 in cross-validation and 0.748 on the leaderboard, is a reasonably strong result for this dataset.

The model is extracting genuine, non-trivial signal from the available physiological and disease-severity features.

REFERENCES

- [1] scikit-learn developers, "Comparing random forests and histogram gradient boosting models." https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_hist_grad_boosting_comparison.html, 2026.
- [2] scikit-learn developers, "1.11. ensembles: Gradient boosting, random forests, bagging, voting, stacking." <https://scikit-learn.org/stable/modules/ensemble.html>, 2024. Section "GradientBoostingClassifier vs HistGradientBoostingClassifier".
- [3] E. Christodoulou, J. Ma, G. S. Collins, E. W. Steyerberg, J. Y. Verbakel, and B. Van Calster, "A systematic review shows no performance benefit of machine learning over logistic regression for clinical prediction models," *Journal of Clinical Epidemiology*, vol. 110, pp. 12–22, 2019.